LABORATORIO DE MACHINE LEARNING · MÓDULO III

Optimizadores: de SGD con Momentum a Adam

Por qué la red no aprendía, y cómo la regla de actualización de los pesos se convirtió en el ingrediente silencioso que hizo posible el deep learning moderno.



SGD Momentum Adam

Teoría — complemento del notebook interactivo

— EL PROBLEMA

¿Qué es, en el fondo, un optimizador?

Una red neuronal aprende ajustando sus pesos **θ** para minimizar una función de pérdida **L(θ)**. El optimizador es, literalmente, la regla que decide:

- → **Dirección** — hacia dónde mover cada peso.
- → **Magnitud** — cuánto moverlo en ese paso.
- → **Memoria** — si usa o no información de pasos anteriores.

REGLA GENERAL DE ACTUALIZACIÓN

$$ \theta_{t+1} = \theta_t - \eta \cdot (\text{algo derivado del gradiente } g_t) $$

Todo lo que distingue a SGD, Momentum y Adam es **qué va dentro de ese paréntesis**. La historia de los optimizadores es la historia de ir mejorando esa expresión, paso a paso, para que la red converja más rápido y de forma más estable.

`η = learning rate · g = gradiente de la pérdida`

PUNTO DE PARTIDA

Descenso de gradiente (SGD)

REGLA DE ACTUALIZACIÓN

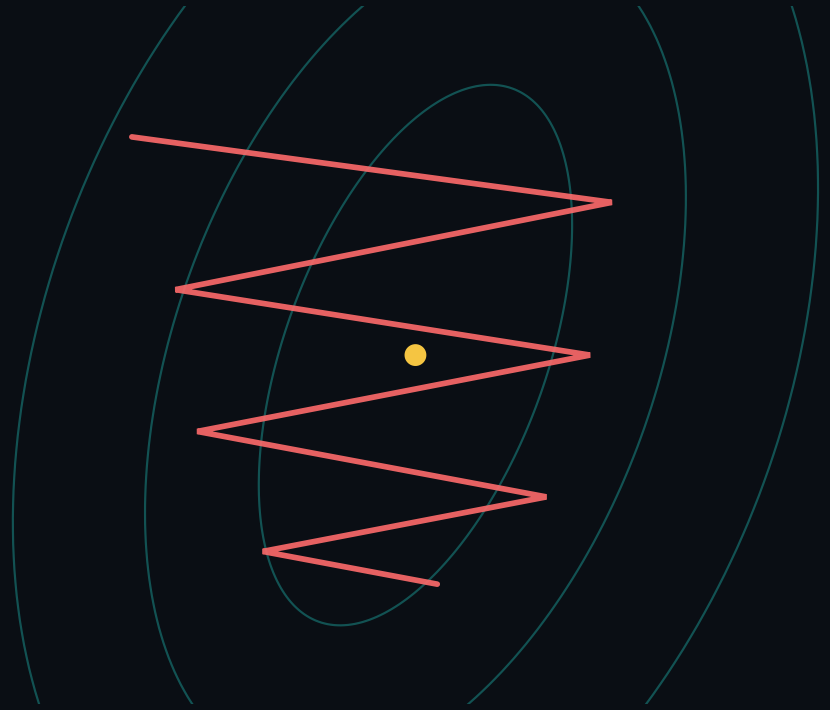
$$ \theta_{t+1} = \theta_t - \eta \, g_t $$

En cada paso, SGD mira **únicamente el gradiente actual** y da un paso proporcional a él, escalado por el learning rate **η**. No tiene memoria de los pasos anteriores.

- → Simple, barato de calcular y fácil de entender.
- → Funciona bien en superficies de pérdida "bien portadas" (convexas, redondeadas).
- × **Es miope:** ignora hacia dónde veníamos moviéndonos.
- × **Un solo learning rate global** para los millones de parámetros de la red.

— EL LÍMITE DE SGD

El dilema del learning rate en un valle angosto



Cuando la pérdida forma un valle angosto y curvo (algo muy común en redes reales), SGD enfrenta un dilema:

- → **η alto** → el paso "rebota" de pared a pared del valle: **zigzagueo**, oscilación e incluso divergencia.
- → **η bajo** → el paso avanza tan poco que la red aprende **demasiado lento**.

No existe un único η que funcione bien en todas las direcciones del espacio de pesos al mismo tiempo.

— PRIMERA EVOLUCIÓN

SGD + Momentum

REGLA DE ACTUALIZACIÓN

$$ v_t = \beta \, v_{t-1} + (1-\beta)\, g_t $$ $$ \theta_{t+1} = \theta_t - \eta \, v_t $$

Idea: en vez de usar el gradiente crudo, acumulamos un **promedio móvil v** de los gradientes recientes — una especie de "velocidad" o inercia.

Analogía física: una bola pesada rodando por el valle. Su inercia hace que:

- → Las oscilaciones perpendiculares al valle **se cancelen** (van en direcciones opuestas y se promedian).
- → El avance **a lo largo** del valle se **refuerce** (es consistente entre pasos, se va acumulando).

Resultado: converge más rápido y con menos zigzagueo que SGD puro, usando el **mismo** gradiente crudo.

— TODAVÍA NO ALCANZA

Un learning rate global sigue sin ser suficiente

Momentum mejora la **dirección** del paso, pero sigue escalando todos los parámetros con el **mismo η**. En una red real, distintos parámetros necesitan pasos de tamaño muy distinto:

- → Parámetros con gradientes **grandes y frecuentes** deberían moverse con pasos **chicos** (para no divergir).
- → Parámetros con gradientes **pequeños y raros** deberían moverse con pasos **grandes** (para no estancarse).

LA PREGUNTA QUE FALTABA RESPONDER

¿Podemos hacer que **cada parámetro tenga, en la práctica, su propio learning rate** — ajustado automáticamente según su propia historia de gradientes?

Esa pregunta es exactamente lo que responden AdaGrad, RMSProp y, finalmente, Adam.

— EL PUENTE

De AdaGrad a RMSProp: aprendiendo a adaptar el paso

2011

AdaGrad

$$ \theta_{t+1} = \theta_t - \dfrac{\eta}{\sqrt{\textstyle\sum g_t^2}+\epsilon}\, g_t $$

Divide el paso por la raíz de la **suma acumulada** de gradientes al cuadrado. Los parámetros con gradientes históricamente grandes reciben pasos más chicos, y viceversa.

LIMITACIÓN

El problema de AdaGrad

$$ \sum g_t^2 \;\longrightarrow\; \infty $$

Como la suma solo **crece**, el denominador se vuelve enorme con el tiempo y el paso efectivo tiende a **cero** — el entrenamiento se frena antes de converger.

2012

RMSProp

$$ v_t = \beta v_{t-1} + (1-\beta) g_t^2 $$

Reemplaza la suma acumulada por un **promedio móvil** de los gradientes al cuadrado. El denominador ya no explota — se adapta a la magnitud **reciente** del gradiente.

LA SÍNTESIS · 2014

Adam = Momentum + RMSProp

1 — PRIMER MOMENTO (DIRECCIÓN, COMO MOMENTUM)

$$ m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t $$

2 — SEGUNDO MOMENTO (ESCALA, COMO RMSPROP)

$$ v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 $$

3 — CORRECCIÓN DE SESGO (M Y V ARRANCAN EN 0)

$$ \hat m_t = \dfrac{m_t}{1-\beta_1^t} \qquad \hat v_t = \dfrac{v_t}{1-\beta_2^t} $$

4 — ACTUALIZACIÓN FINAL, POR PARÁMETRO

$$ \theta_{t+1} = \theta_t - \dfrac{\eta}{\sqrt{\hat v_t} + \epsilon}\, \hat m_t $$

— POR QUÉ GANÓ

Por qué Adam se volvió el estándar de la industria

- → **Adapta el paso por parámetro** — cada peso tiene, en la práctica, su propio learning rate efectivo.
- → **Hereda la inercia de Momentum** — atraviesa valles angostos sin zigzaguear.
- → **No explota como AdaGrad** — usa promedios móviles, no sumas acumuladas.
- → **Robusto a la elección de η** — converge razonablemente bien incluso sin un ajuste fino de hiperparámetros.

EN UNA FRASE

Adam le da a cada parámetro **su propia "prudencia"**: avanza rápido donde el terreno lo permite, y con cautela donde el gradiente ha sido errático.

Por eso es el valor **por defecto** en la enorme mayoría de papers, frameworks (PyTorch, TensorFlow/Keras) y proyectos de deep learning modernos.

— EL OTRO PROBLEMA

Por qué una red profunda deja de aprender

Backpropagation calcula el gradiente de una capa temprana **multiplicando** las derivadas de todas las capas que vienen después (regla de la cadena):

GRADIENTE DE UNA CAPA TEMPRANA (L CAPAS)

$$ \frac{\partial \mathcal{L}}{\partial \theta_1} \;\propto\; \prod_{l=1}^{L} \sigma'(z_l) $$

La derivada de la sigmoide, **σ'(z) = σ(z)(1−σ(z))**, tiene un **máximo de solo 0.25** y cae cerca de 0 cuando la neurona está saturada.

- × Multiplicar 4 o 5 números ≤ 0.25 da un producto **diminuto**.
- × Las **primeras capas** reciben una señal de gradiente casi nula.
- × Quedan casi **congeladas** en su inicialización aleatoria — la red no aprende representaciones útiles ahí.

— LA SOLUCIÓN PARCIAL

Cómo Adam **mitiga** (pero no resuelve) el problema

El paso de Adam se normaliza dividiendo por $\sqrt{\hat{v}_t}$, la magnitud típica reciente del gradiente **de ese parámetro**:

$$ \Delta\theta \;=\; \dfrac{\hat m_t}{\sqrt{\hat v_t} + \epsilon} $$

Si un parámetro tiene gradientes consistentemente chicos, $\hat{v}_t$ **también es chico** — y el cociente se reescala hacia un paso razonable, en vez de colapsar a cero como en SGD.

IMPORTANTE

Adam **no cambia la causa raíz**: la saturación de la sigmoide sigue produciendo gradientes crudos diminutos.

Por eso, en la práctica moderna, el vanishing gradient se ataca con una **combinación** de soluciones:

- → **ReLU** y variantes (derivada no se satura para z>0).
- → **Inicialización** He / Xavier.
- → **Batch / Layer Normalization.**
- → **Conexiones residuales** (ResNets).
- → … y un optimizador adaptativo como **Adam**.

RESUMEN

Tres optimizadores, tres formas de dar el paso

OPTIMIZADOR	USA MEMORIA (DIRECCIÓN)	ADAPTA POR PARÁMETRO (ESCALA)	SENSIBILIDAD A H	VANISHING GRADIENT
SGD	No	No	Alta	Sin mitigación
Momentum	Sí	No	Media	Mitigación leve
Adam	Sí	**Sí**	**Baja**	Mitigación notable

— EN LA PRÁCTICA

Esto es lo que vimos en el notebook interactivo

- → Trayectorias de SGD / Momentum / Adam sobre un valle angosto (función tipo Rosenbrock).
- → Una red neuronal **desde cero, con NumPy**, entrenada sobre el dataset público *Iris* (flores — versicolor vs. virginica).
- → La norma del gradiente **por capa**, capturada en vivo durante el entrenamiento.

LO QUE SE COMPROBÓ EMPÍRICAMENTE

Con **SGD** puro, el gradiente de la primera capa quedó **órdenes de magnitud** por debajo del de la última capa. Con **Adam**, esa brecha se redujo y la red convergió más rápido y de forma más estable.

GRACIAS

De un gradiente que desaparece a un optimizador que se adapta

Preguntas, dudas o el notebook interactivo — todo queda abierto a discusión.